



Symphony6 SSTN Configuration Document For OEM

Version: 1.1

Revision Date: 2025-6-23

Montage LZ Proprietary and Confidential

Change History

Version	Date	Author	Comment
1.0	May. 29, 2025	Montage-LZ	The first draft
1.1	Jun.23,2025	Montage-LZ	Correct typo according to review

Nagra Confidential

Content

1	Introduction.....	4
2	Symphony6 TEE brief.....	4
3	Specific needs for TEE.....	5
3.1	Chipset configuration for TEE	5
3.2	Device provision for TEE	6
3.2.1	TEE segmentation	6
3.2.2	Anti-rollback	7
3.3	Memory map	8
3.4	Firewall configuration and TEE configuration	8
3.5	Secure storage	10
3.5.1	TEE relevant software protection.....	10
3.5.2	TEE data storage protection	12
3.6	Secure Time.....	12
3.7	Secure upgrade.....	13
3.7.1	TEE-OS upgrade	13
3.7.2	TA upgrade	14
3.8	Low power modes.....	14
3.8.1	STR mode	15
3.8.2	Standby mode.....	15
3.9	Merge tool	15
4	Specific needs for SMP and OP&C	15
4.1	Chipset configuration for SMP	15
4.2	Chipset configuration for OP&C.....	18
4.3	Additional HW components for SMP and OP&C.....	18
5	Acronyms/Terms	18

Related Documents

- [1] SPE_NOCS3x_SSTN version 2.2.0 of Montage_NOCS3.1ProfileB_SpecificationKit-01.04.00.
It is from NAGRA.
- [2] LZ_Linux_Power_Userguide.pdf. It is released in Montage SDK kit.
- [3] Symphony6 NAGRA Flash Merge Introduction.pdf. It is released with merge tool by Montage.

1 Introduction

This document is aimed to provide a user guide for OEMs to integrate Symphony6 chipsets correctly into their devices, configure them and use them in a way that their device can comply with NAGRA SSTN specification requirements in document ([see reference \[1\]](#)).

It contains the following sections:

- Symphony6 TEE brief
- TEE software segmentation and versioning
- Secure storage
- Secure time
- Secure upgrade
- The typical SMP use cases
- CUR enforcement
- Firewall settings

2 Symphony6 TEE brief

- TEE OS: OP TEE which is GlobalPlatform (GP) compatible
- Hardware support for TEE
 - Based on ARM V8 TrustZone to provide isolation of the TEE from the rich OS in hardware
 - Support TAs(trusted application) isolation using MMU
 - I/O and Memory/Register isolation using Montage private AXI/AHB/APB firewall

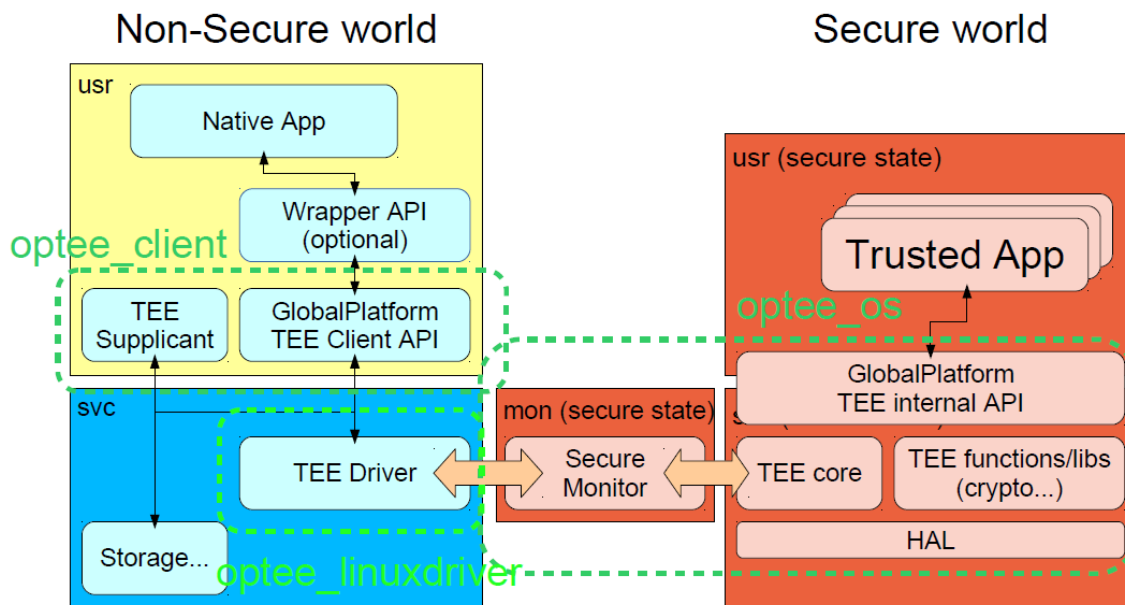


Figure 1 Symphony6 OP TEE Brief

3 Specific needs for TEE

3.1 Chipset configuration for TEE

When Montage provides Symphony6 chipsets to OEMs, chipsets are in production state for TEE. All security of functional feature, like TEE Secure Boot, Version Checking, Authenticity or Secrecy protection, is activated by Montage. So OEMs don't need to do anything about chipset configuration for TEE.

3.2 Device provision for TEE

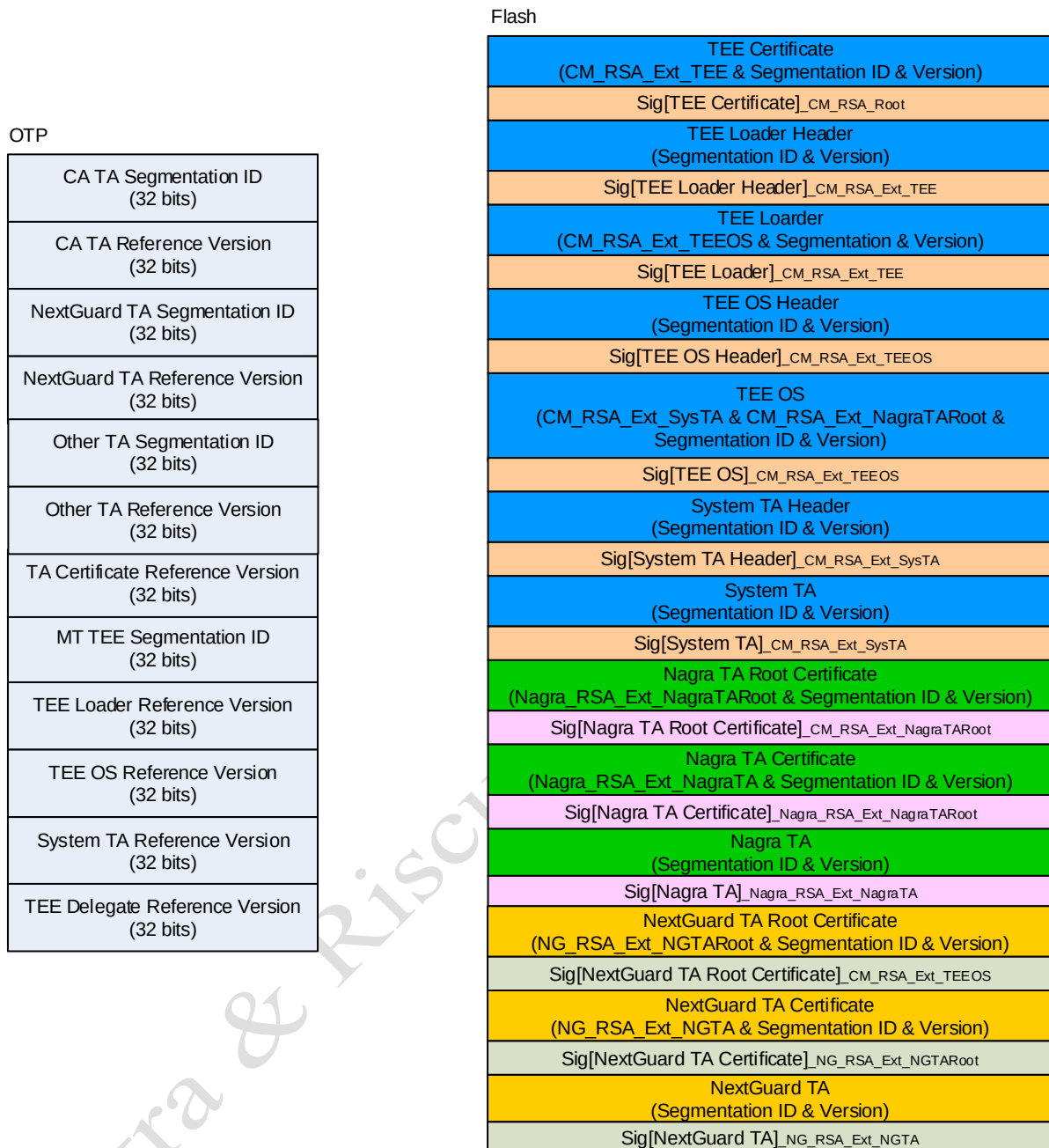


Figure 2 TEE software segmentation and versioning

3.2.1 TEE segmentation

CA TA Segmentation ID: 32 bits, can be assigned a different value for each Device model, in Nagra case, it is used for Nagra TA Segmentation ID. In Nagra OTP fuse map, it is OTP_CA_TA_MSID.

NexGuard TA Segmentation ID: 32 bits, it can be used for NexGuard TA Segmentation ID, in OTP fuse map, it is OTP_NexGuard_TA_MSID.

MT TEE Segmentation ID: 32 bits, can be assigned a different value for each Chipset family, it can be used for Montage-LZ private TA and TEE, in OTP fuse map, it is OTP_TEE_MSID

Other TA Segmentation ID: 32 bits, it can be used for other TA Segmentation ID, other means this kind of TAs are not published by CA(for example Nagra), NexGuard or Montage-LZ, in OTP fuse map, it is OTP_Other_TA_MSID.

Table 1 OTP Segmentation Configurations for OEMs

Name	Bit Offset	Bit Length	Stage2 Value	Description
OTP_CA_TA_MSID	0x1CFE0	32	Nagra Define	Nagra TA MSID
OTP_SWL_CA_TA_MSID	0x1EB42	3	3b111	Write lock for Nagra TA MSID
OTP_NextGuard_TA_MSID	0x1CF60	32	Nagra Define	NextGuard TA MSID
OTP_SWL_NGTA_MSID	0x1ED22	3	3b111	Write lock for NextGuard TA MSID
OTP_TEE_MSID	0x1E960	32	Montage Define	Montage-LZ private TA and TEE MSID
OTP_SWL_TEE_MSID	0x1EB65	3	3b111	Write lock for OTP_TEE_MSID
OTP_Other_TA_MSID	0x1D800	32	Third party Define	The MSID of Tas are not published by Nagra, NextGuard or Montage
OTP_SWL_CAVReserve d15	0x1ED0E	3	3b111	Write lock for OTP_Other_TA_MSID

NOTE:

- 1) Application should call API: `mt_unf_otp_write()` to fuse fields above.

3.2.2 Anti-rollback

CA TA Reference Version: 32 bits for CA TA versioning, in Nagra case, total 32 bits can be used by Nagra Tas, in the OTP fuse map, it is `OTP_CA_TA_Version_REF`.

TA Certificate Version: 32 bits reference version used for all Tas Certificate, Nagra only use the lower 16 bits to store their TA Certificates reference version, and the higher 16 bits can be used for other TA Certificate including the one of NexGuard TA if needed. In OTP fuse map, it is `OTP_TA_Certificate_Ver_REF`.

NexGuard TA Reference Version: 32 bits dedicated for NexGuard TA Certificate versioning, in OTP fuse map, it is `OTP_NexGuard_TA_Ver_REF`.

MT TEE(Loader/OS/System TA/TEE Delegates) Reference Version: there are four 32 bits for Montage TEE software versioning. In OTP fuse map, they are `OTP_Version_TEEloader_REF`, `OTP_Version_TEEOS_REF`, `OTP_Version_SysTA_REF` and `OTP_TEEDelegates_Version_REF`.

Other TA Reference Version: 32 bits dedicated for other TA Certificate versioning, other means this kind of Tas are not published by CA(for example Nagra), NexGuard or Montage-LZ, in OTP fuse map, it is `OTP_Other_TA_Ver_REF`.

NOTE:

- 1) All the OTP of Reference Version should be programmed by Symphony6 Secure boot mechanism. It means usually OEMs should not program them, otherwise it will cause boot failure. So we do not provide OTP fuse address about Reference Version here.

3.3 Memory map

There are only one typical memory maps of 512M case. If there is more than 512M DDR memory, you only can configure it to be non-secure system area. OEMs should only operate non-secure RAM. AV CPU Firmware is in 0~10M, TEE is in 10M~46M.....

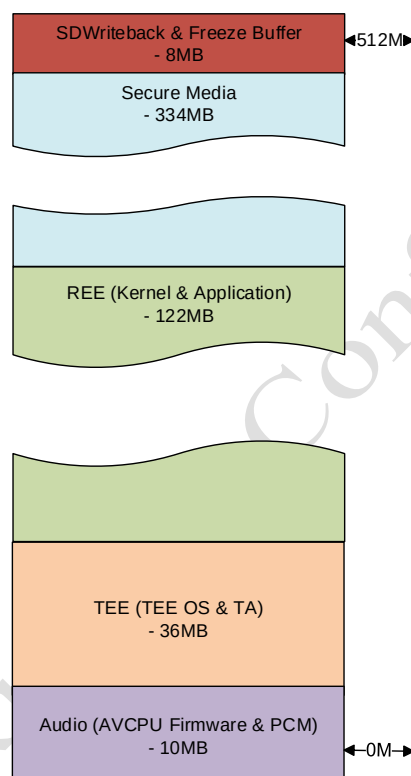


Figure 3 Memory Layout for 512MB DDR

3.4 Firewall configuration and TEE configuration

In SCS stages, Symphony6 will set firewall configuration (in SCS_Params) and TEE configuration (in TEE_loader_header). The firewall and TEE configuration parameters are from Montage, and they are very important for SMP and OP&C. Please don't modify it; otherwise the board boot will fail.

Table 2 the memory map of SCS_Params.

Name	Size	Comment
Init_Flag	4 Bytes	Magic of flash image
Scrambling_Activation	4 Bytes	Enable the DRAM scrambling
Scrambling_Mode	4 Bytes	DRAM scrambling mode, available only when Scrambling_Activation is enabled. 0x00000000: re-new the scrambling key without key preservation.

		0x55555555: re-new the scrambling key
Default_Area_Min_Size	4 Bytes	Minimal size of default area.
SCS_DTE_BootCode_Offset	4 Bytes	Flash offset of DTE_BootCode
SCS_DTE_BootCode_LoadAddr	4 Bytes	DRAM loading address of DTE_BootCode
SCS_Total_Area_Size	4 Bytes	Total size of flash image
DTEEnc_AfterSign_Ena (Flash_Protection_Activation)	4 Bytes	The encryption for DTE Boot Code is enabled in the case of signing on plaintext.
Reserved	32 Bytes	
EDCK (FlashProtectionKey)	16 Bytes	The encrypted DTE_BootCode encryption key, in Nagra case, it is same as FlashProtectionKey.
AVCPU_Ena	4 Bytes	The flag to indicate if configure firewall in SCS for AVCPU start up. 0x0: AVCPU is disabled; others: AVCPU is enabled.
Reserved	4 Bytes	
Ext_RAM_Params	512 Bytes	The parameters for external RAM initialization
Firewall_Config	1432 Bytes	Firewall configuration after SCS finished(including the 32 bytes HMAC-SHA256 signature at the end)
MSID_Ext	4 Bytes	
MSID_Mask	4 Bytes	
VER_Ext	4 Bytes	
VER_Mask	4 Bytes	
Signature	256 Bytes	Signature of SCS_Param. Signed by CA_RSA_Ext

Table 3 the memory map of TEE_Loader_Header

Name	Size	Comment
Block ID	1 Bytes	Magic number to identify TEE_Loader_Header
Reserved	3 Bytes	Filled with 0
TEE_Loader_Offset	4 Bytes	Flash offset of TEE_Loader
TEE_Loader_LoadAddr	4 Bytes	Loading address in DRAM
TEE_Loader_Size	4 Bytes	Size of TEE_Loader
ETLK_1(TEE Type Chipset Family)	16 Bytes	KeyLadder first input of TEE Loader encryption
ETLK_2(Provider ID)	16 Bytes	KeyLadder second input of TEE Loader encryption
ETLK_3(Encrypted TEE Loader encryption Key)	16 Bytes	KeyLadder third input of TEE Loader encryption
TEE_Config	1968 Bytes	Configuration parameters for TEE(including the setting of TEEDataCfgChk registers)
MSID_Ext	4 Bytes	
MSID_Mask	4 Bytes	
VER_Ext	4 Bytes	
VER_Mask	4 Bytes	
Signature	256 Bytes	Signature of TEE_Loader_Header. Signed by CM_RSA_Ext_TEE

3.5 Secure storage

3.5.1 TEE relevant software protection

This following diagram shows how to realize the TEE software protection, the each software component (TEE Loader/TEE OS/TA) is encrypted by the respective protection key which is owned by the software provider, and the protection key is encrypted by the key derived from the key ladder whose root key is chipset family universally embedded in SoC, this encrypted protection key is stored in flash as the last input of the key ladder to generate the protection key, and it is signed together with other input constants by specific the external TEE RSA Root Key from the software provider. OEMs don't need to understand detail about them, since Montage and TA provider will provide all correct images.

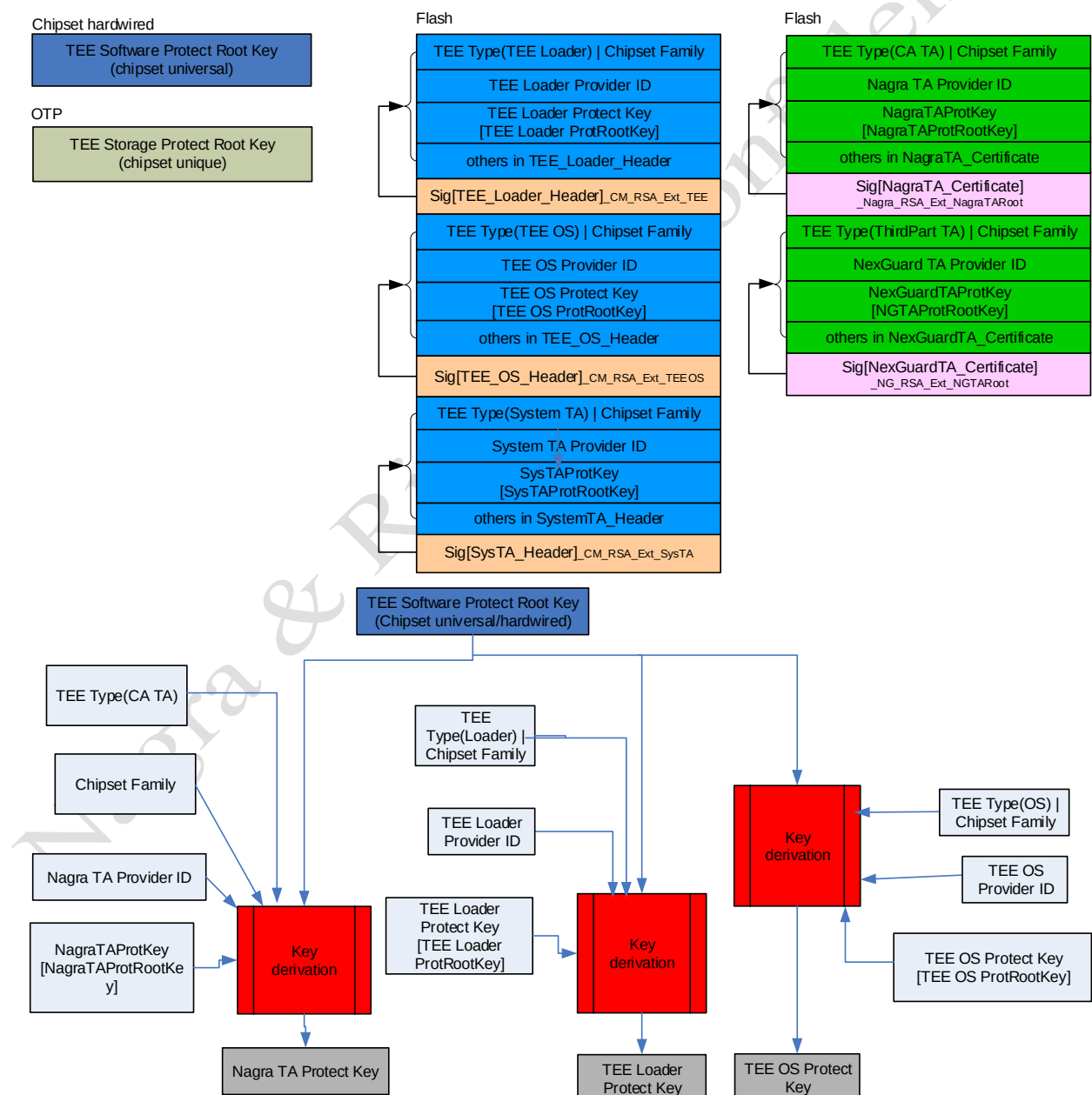


Figure 5 TEE software protection key derivation

The following tables define the structure of TA Root Certificate and TA Certificate:

Table 3 Structure of TA Root Certificate

Name	Owner	Size	Comment
RSA_Ext_TA_Root	TA Provider	256 Bytes	The external RSA public key to authenticate TA Certificate
TA_RootCert_MSID_Ext	Montage-LZ	4 Bytes	The segmentation id of TA Root Certificate.
TA_RootCert_MSID_Mask	Montage-LZ	4 Bytes	The mask of TA Root Certificate segmentation id
TA_RootCert_VER_Ext	TA Provider	4 Bytes	The version of TA Root Certificate.
TA_RootCert_VER_Mask	TA Provider	4 Bytes	The mask of TA Root Certificate version
Signature	Montage-LZ	256 Bytes	Signature of TA Root Certificate, signed by CM_RSA_Ext_TA_Root

Table 4 Structure of TA Certificate

Name	Owner	Size	Comment
RSA_Ext_TA	TA Provider	256 Bytes	The external RSA public key to authenticate TA
Constant1	Montage-LZ	16 Bytes	Constant as the 1 st input of key ladder to generate TAProtRootKey, given by TEE Type and Chipset Family
Constant2	Montage-LZ	16 Bytes	Constant as the 2 nd input of key ladder to generate TAProtRootKey, given by TA Provider ID
TAProtKey[TAProtRootKey]	TA Provider	16 Bytes	TAProtKey encrypted by TAProtRootKey
TA_Certificate_MSID_Ext	Montage-LZ	4 Bytes	The segmentation id of TA Certificate.
TA_Certificate_MSID_Mask	Montage-LZ	4 Bytes	The mask of TA Certificate segmentation id
TA_Certificate_VER_Ext	TA Provider	4 Bytes	The version of TA Certificate.
TA_Certificate_VER_Mask	TA Provider	4 Bytes	The mask of TA Certificate version
Signature	TA Provider	256 Bytes	Signature of TA Certificate, signed by RSA_Ext_TA_Root

The following table gives the rule how to define TEE Type, Chipset Family and TA Provider.

Table 5 Definition of TEE Type, Chipset Family and TEE Provider

Name	Size	Comment
TEE Type	8 Bytes	High four bytes are the fixed symbol data, and low four bytes are types 64h FFFFFFFF00000000: AuxCode 64h FFFFFFFF00000001: TEE Loader 64h FFFFFFFF00000002: TEE OS 64h FFFFFFFF00000003: System TA 64h FFFFFFFF00000004: CA TA 64h FFFFFFFF00000005: DRM TA 64h FFFFFFFF00000006: Other TA

Chipset Family	8 Bytes	High four bytes are fixed symbol data, and low four bytes are chipset family data. . 64h FFFFFFFF1100000000: Symphony6 ...
TEE Provider	16 Bytes	High four bytes are the fixed symbol data, and low twelve bytes is TEE software component provider ID. 128h FFFFFFFF22000000000000000000000000000000: Montage-LZ 128h FFFFFFFF22000000000000000000000000000001: Nagra 128h FFFFFFFF22000000000000000000000000000002: NexGuard ...

3.5.2 TEE data storage protection

About storage protection, we follow the standard procedure in OP TEE. We deploy a chipset unique key (TEE Storage Protect Root Key) in OTP for the purpose of platform binding in level of chipset.

3.6 Secure Time

Symphony6 has a hardware timer which has 64 bits counter with the cascade of two 32bit counters, the input clock is 1M Hz, which gives a very long time enough in a power-on lifecycle, and we can configure this timer to TEE access only by AXI firewall, so it can be used as the secure time.

Only TEEOS can write to secure timer and no any TA can write to it, secure time is readable by REE and other 3rd TA.

But Montage only provides interface for TEE to read secure time now, OEMs cannot read it in REE.

3.7 Secure upgrade

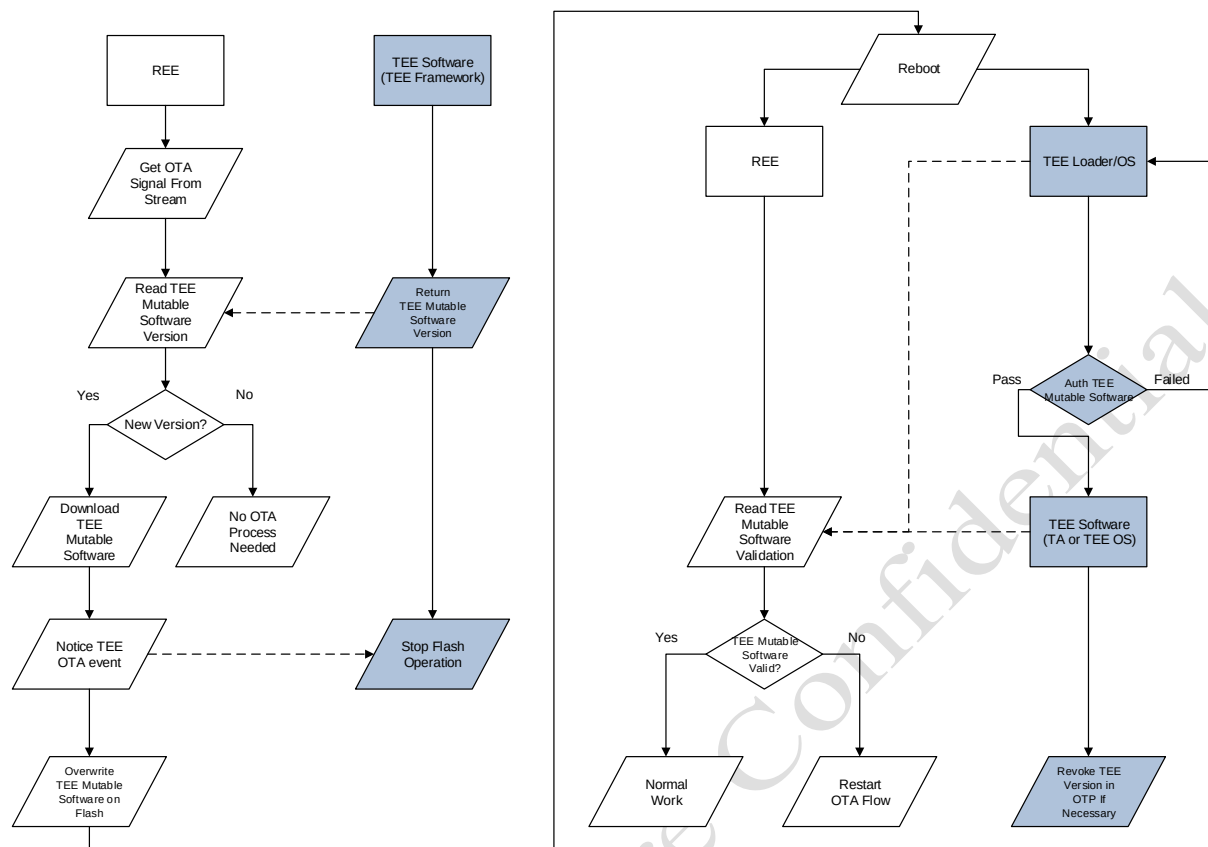


Figure 6 TEE mutable software upgrade flow

In this solution, TEE Loader is taken as an immutable component which is not upgradable.

NOTE:

- 1) Current Nagra TA does not do the TEE version revoking. The Nagra TA version revoking is done by TEE OS when the upgrade is finished and the authentication is successfully passed.
- 2) OTP Reference Version should be programmed by Symphony6 Secure boot mechanism. It means usually OEMs should not program them, otherwise it will cause boot failure.

System TA or other third part TA has its individual version, all system TAs share one 32 bits version with each TA occupying some bit field, so that each TA can be updated without affecting other TAs.

In this process, if there is an error in programming the flash that the TEE software is corrupted, thus it leads to a failed authentication in TEE Loader during the next power up, in this case the TEE Loader will notify the REE that the secure upgrade is failed; REE can restart a new OTA process.

In this diagram, 'revoke' means updating the new version into the OTP. But OEMs don't need to program the OTP, since it will be done by TEE Loader or TEE OS!

Montage will provide some reference code about TEE-OS and TA upgrade for OEMs.

3.7.1 TEE-OS upgrade

After getting TEE-OS image successfully in upgrade APP which is developed by project owner, please just program

the new one to fixed flash offset (It is defined by project, usually it is 0x500000) by flash write API. After rebooting the board, the new TEE OS will be verified in TEE-loader, if TEE-OS is verified success, it will be boot; otherwise DTEBOOT will get a failed parameter.

When DTEBoot triggers TEE-loader to boot TEE-OS image with command “smc 0x42000001 0x3100000 0x100000 0x3000000” (Please note there is a tee dtb which is programmed to flash too, and TEE-OS will use it during booting). there is an environment named “teeosboot”. Please check it and do TEE-OS upgrade again if “teeosboot” is equal fail. Montage provides a reference boot arguments in boot script about it.

If developer has any question about it, please contact Montage for support.

Below is the main process about upgrade TEE-OS:

- 1, Board gets TEE-OS upgrade signal and enter upgrade APP. (It maybe a flag in flash or some other signal defined by project)
- 2, Upgrade APP gets TEE-OS by OTA program, USB disk or some other way defined by project.
- 3, Upgrade APP burns new TEE-OS image to flash to override old one by flash write API. (The TEE-OS flash offset address should correspond to project design; usually it is 0x500000)
- 4, Upgrade APP reboots board, and DTEboot will boot TEE-OS and check parameter “teeosboot”. If TEE-OS boot is fail, please do step 1-4 again.

Boot TEE-OS reference command in DTEboot (DTEboot demo project and boot script are included in Montage SDK develop kit):

```
smc 0x42000001 0x3100000 0x100000 0x3000000
if test "$teeosboot" = "fail" ; then
    echo entry teeos upgrade OTA
    load_uimage 0 0x2000000 0x2000000 0x07600000
    dcache flush;
    bootm 0x07600000 - 0x03000000
else
    echo "*load kernel"
    load_uimage 0 0x3000000 0x2000000 0x07600000
    dcache flush;
    bootm 0x07600000 - 0x03000000
fi
```

3.7.2 TA upgrade

When OEMs wants to upgrade TA, then just override the old one after getting it. And the TA will be verified when TEE-OS boots it, if it is invalid, OEMs should upgrade it again.

Below is the main process about upgrade TA:

- 1, OEMs get TA upgrade signal
- 2, OEMs get TA by OTA
- 3, OEMs override old one by command “cp -f new_TA.image /lib/optee_armtz/”

Usually, Montage puts TA images under “/lib/optee_armtz/”

- 4, Boot TA and check return value, if TA boot is fail, do step 2-4 again.

3.8 Low power modes

Montage provides two low power modes in Symphony6. One is STR mode, it is warm boot; the other one is standby mode, it is code boot. Two modes have the similar configuration process, but different entry command.

Please refer to introduce document ([see related reference \[2\]](#)) for more detail about low power operation.

Montage will provide reference code for two modes.

3.8.1 STR mode

There is a DDR protection mechanism in STR mode which complies with NAGRA SSTN specification requirements in document ([see reference \[1\]](#)). OEMs only need to run below command to enter STR mode in REE side. If developer has any question about it, please contact Montage for support.

```
system("echo mem >/sys/power/state")    //Enter STR mode
```

3.8.2 Standby mode

It has the similar configure process with STR mode, but the entry command is a function “MT_UNF_PMOC_SwitchSystemMode”, and it doesn’t support any DDR protection.

3.9 Merge tool

Montage provides a merge tool to OEMs to generate “all.img”, which is sent to NAGRA for production SCS image. Please refer to introduce document ([see reference \[3\]](#)) for details about the tool.

4 Specific needs for SMP and OP&C

Symphony6 implements SMP and OP&C by hardware modules and PTAs in TEE OS. Firewall and TEE configuration are important for SMP and OP&C, please see “3.4 Firewall configuration and TEE configuration” for detail.

4.1 Chipset configuration for SMP

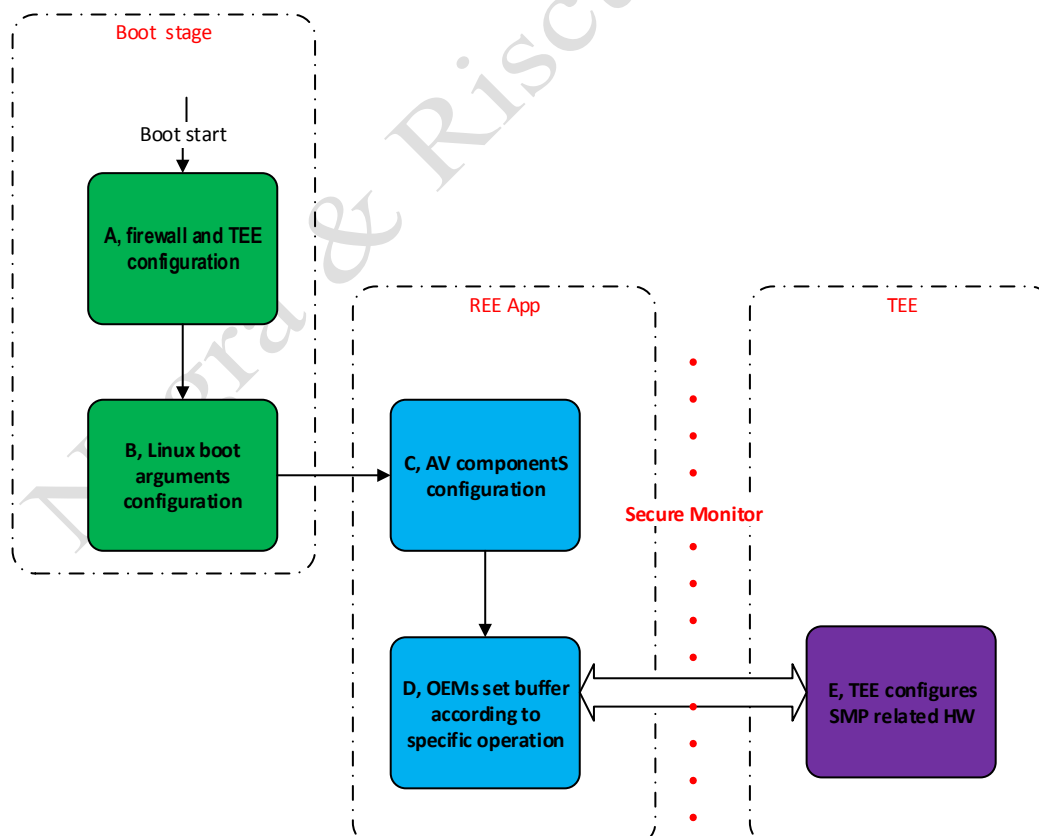


Figure 7 SMP configuration overview flow

The above figure shows SMP configuration flow, it contains five steps.

A, firewall and TEE configuration:

Montage will provide firewall and TEE configuration to OEMs to program to flash.

B, Linux boot arguments configuration:

Montage will provide a recommend boot arguments configuration file, and OEMs can't modify anything about SMP configuration. If OEMs want to modify boot arguments, please must check it with Montage.

DRAM 512M case

```
#DRAM 512M demo configuration
#TEE, AVCPU is fixed in SCS booting stage
#PCM should be fixed to 6M-10M
#AV should be 254M- 442M
setenv pip_mmz mmz=pip, 0, 442M, 62M
setenv memargs mmz=pcm, 0, 6M, 4M mmz=av, 0, 254M, 188M $pip_mmz $dfb_mmz mmz=ott0, 0, 170M, 16M
mmz=ddr, 0, 190M, 68M
```

Please note, we modify “pvr0” area to “ott0” to support OTT SMP, and remove “pvr1” area, since PVR will use dynamic DDR range allocated by low level driver in Symphony6 platform.

C, AV components configuration:

There are several SMP related AV components in chipset, such as Audio, Video, PVR, AV CPU. Montage also provides the demo configuration and init source code to OEMs. When OEMs want to modify anything about them in APP init stage, please refer to related documents: LZ_Linux_AVPlay_Userguide, LZ_Linux_Display_Userguide, LZ_Linux_PVR_Userguide.pdf, and confirm with Montage.

D, OEMs set buffer according to specific operation

Different operations need to be configured with different protected media buffer types. There are several protected media buffer types: video, audio, record, OTT, replay. Montage defines an enumeration, structure and API in mt_sec_ext.h to represent them. When OEMs implement SMP, please set the buffer according to specific operation. Please refer to section 3.3 to confirm the valid range of each type of buffer.

512M case:

The range of audio buffer is dynamical address from our API.

The range of video buffer is from 254M to 442M.

The range of record buffer is configurable dynamically, you can get DDR range by our API.

The range of OTT buffer is configurable, for example: from 170 M to 186M.

```
typedef enum {
    MB_VID = 1, /**The buffer for main video. */
    MB_AUD, /**The buffer for audio. */
    MB_REC, /**The buffer for record channel 0. */
    MB_REC_1, /**The buffer for record channel 1. */
    MB_REC_2, /**The buffer for record channel 2. */
    MB_REC_3, /**The buffer for record channel 3. */
    MB_OTT, /**The buffer for OTT. */
    MB_OTT_1, /**The buffer for OTT. */
    MB_REP, /**The buffer for replay. */
    MB_SUB_VID, /**The buffer for sub video. */
    MB_REENC_OTT, /**The buffer for re-encrypt OTT. */
    MB_REENC_OTT_1, /**The buffer for re-encrypt OTT.1. */
} MBType;
```

```
typedef struct _MediaBufferInfo {
    mt_u32 start; /* start physical address of buffer */
    mt_u32 size; /* byte length of buffer */
    MBType type; /* buffer type */
} MBInfo;

/*!
 * \brief: set media buffer to the list to be protected.
 * NOTE: called by user.
 *
 * \param[In] addrPhy: physical start address if this buffer.
 * \param[In] size:
 * \param[In] type:
 *
 * \return TRUE for success; FALSE for failed.
 */
TBoolean mtSecSetProtectBuffer(mt_u32 addrPhy, mt_u32 size, MBType type);
```

D.1, Open DVB session

When opening a DVB session, OEMs need to configure protected media buffer for audio and video by API `mtSecSetProtectBuffer`. The dynamic audio protected buffer is allocated by our low level driver, and you can get from our APIs.

```
mtSecSetProtectBuffer(dynamic address, dynamic size, MB_AUD);
mtSecSetProtectBuffer(0xFE00000, 0xBC00000, MB_VID);
```

D.2, Open OTT session

When opening a OTT session, OEMs need to configure protected media buffer for OTT, audio and video by API `mtSecSetProtectBuffer`.

```
mtSecSetProtectBuffer(dynamic address, dynamic size, MB_AUD);
mtSecSetProtectBuffer(0xFE00000, 0xBC00000, MB_VID);
/* If SMP is enabled, please configure below protected media buffer for OTT. */
mtSecSetProtectBuffer(0xAA00000, 0x1000000, MB_OTT);
```

D.3 Open recording

When opening a recording operation, OEMs need to configure protected media buffer for record buffer by API `mtSecSetProtectBuffer`

```
mtSecSetProtectBuffer(stStatus.u32BufPhyAddr, stStatus.u32BufSizeAlign, MB_REC);
// Get stStatus from API MT_UNF_DMX_GetRecBufferStatus
```

D.4 Open replaying

When opening a replaying operation, OEMs need to configure protected media buffer for record buffer by API `mtSecSetProtectBuffer`. The data flow is different with other operations. When ciphertext data is decrypted by M2M engine, the plaintext data should be injected into DMX ES buffer. So OEMs need to protect both plaintext data and DMX ES buffer.

```

/* Step1, configure DMX ES buffer is a protected OTT buffer */
#define DEMUX_TS_BUF_SIZE 0x100000
MT_UNF_STREAM_BUF_S dmxBuf = {0};
mt_u32 phyAddrTsBuf = 0;
/*
When call MT_UNF_DMX_CreateTSBuffer, you can get tsBufHandle, please refer to doc
Symphony_Linux_DEMUX_Userguide for more detail about DMX operation
*/
ret = MT_UNF_DMX_GetTSBufferEx(tsBufHandle, 188, &dmxBuf, &phyAddrTsBuf, 10);
if (ret != MT_SUCCESS) {
    return NV_SPR_ERROR;
}
mtSecSetProtectBuffer(phyAddrTsBuf, DEMUX_TS_BUF_SIZE, MB_OTT);

/* Step2, configure a plain text DRAM buffer is a protected OTT buffer */
tsBufferSize = 0x40000; /* 256K aligned for SMP-ON */
tsBuffer = (TUnsignedInt8 *)mmz_malloc("ddr", tsBufferSize);
tsBufferPhy = vir2phy(ts->tsBuffer);
mtSecSetProtectBuffer(tsBufferPhy, tsBufferSize, MB_OTT);

/* Step3, configure audio and video buffer are protected buffers */
MT_UNF_DMX_GetChannelHandle(ts->playDmxId, audPid, &audioChannel);
MT_UNF_DMX_GetEsBuffPhyAddr(audioChannel, (ulong *)&esAudioBuffPhyAddr, &esBuffSize);
mtSecSetProtectBuffer(esAudioBuffPhyAddr, esBuffSize, MB_AUD);
mtSecSetProtectBuffer(0xFE00000, 0xBC00000, MB_VID);

```

4.2 Chipset configuration for OP&C

OP&C is controlled by Symphony6 TEE side with NAGRA usage rules. And the operation process is not visible to the OEMs. So OEMs don't need to do any configuration.

4.3 Additional HW components for SMP and OP&C

In Symphony6, from software point of view, there are 3 hardware mechanisms used for SMP and OP&C: AHB/APB Firewall, CID Mechanism and AXI Firewall. They are configured in TEE side by calling related APIs: `mtSecSetProtectBuffer`, `SMPC_Open`, `SMPC_RegisterSecureMemory` and `SMPC_Close`. But `SMPC_Open`, `SMPC_RegisterSecureMemory` and `SMPC_Close` are wrapped in SEC APIs, so OEMs don't need to know the details.

5 Acronyms/Terms

Name	Description
SOC	System On Chip
SCS	Secure Chipset Startup
TEE	Trusted Execution Environment
REE	Rich Execution Environment
OTA	Over the air
OEM	Original Equipment Manufacturer
SMP	Secure Media Path

OP&C	Output Control
CM	Chipset Manufacturer
STBM	Set-Top-Box Manufacturer
STR	Suspend to RAM
SDK	Software Development Kit

Nagra & Riscure Confidential